Week 1: Network

Network

- A structure linking devices together for the purpose of communication
- A device is a node, and a physical connection is a link

How does the internet work?

- Sequence of bits sent through connections
- Network communication requires protocols, a common language

Protocols

- IP (internet protocol)
  - Consistent addressing of entities on the internet
- BGP (broader gateway protocol)
  - Finding best possible route on internet
- TCP (Transmission Control Protocol)
  - Error free delivery of data streams
- SMTP (Simple Mail Transfer Protocol)
  - Sending and receiving emails
- FTP (File Transfer Protocol
  - Sending and receiving files

Models

- TCP/IP
  - Every device uses this model
  - Tolerate unreliable sub-networks and the protocol guarantees proper transmission of data since links can't.
  - **How it guarantees**:
    - Transmission not needing guarantees the data will arrive.
    - UDP
      - User Datagram Protocol
      - Arrives faster because it removes error detection or correction overheads
      - Packets that are not received are forgotten about
      - Prefer real time than delayed
  - **Layers**:
    - Application Layer
      - HTTP, TLS, DNS
      - Protocols on what the format the data should be in
    - Transport layer
      - TCP, UDP
      - Protocols convert the stream to and from packets
      - Detect and fix packets, lost or corrupted
    - Internet layer
      - IP(v4, v6)
      - Transmitting a single packet from source to destination across network
    - Network access layer
      - Ethernet, Wireless LAN
      - How a single packet is transmitted across a single link in the network

Communication Protocols

- Rules and procedures for control timing and data format
- Structure
  1. Client -> Signal
  2. Server -> Identify
  3. Server -> Transmit
  4. Server -> Retransmit
  5. Client -> Detect
  6. Client -> Signal
  7. Server -> Terminate

Client-Server Architecture

- Server address is known and stores info
- Client sends a request for info to server via (HTTP, FTP, SMTP)
- Server then transmit to client

Advantages:

- Multiple clients use a single server
- New clients can join system without registration
- Single central source

Disadvantages:

- Single point of failure
- Too many clients -> overloads server
- Multiple servers with same content exist, need to keep synchronised

Week 2: HTML

HTML describes the semantic content of a web page and logical relations between content structure

Basics

- Hyper Text Markup Language
- Is a tree data structure
    - Document Object Model

Philosophy

1. Interoperability
    - Renders on wide variety of browsers
2. Graceful Error Recovery
    - Small errors shouldn't stop page from rendering
3. Backwards Compatible
    - New Features shouldn't break web
4. Prioritise User
    - User experience should be best
    - User -> WebDev -> Browser -> Theorist
5. Separation of concerns
    - Describe type of information not how it displays

Week 3: CSS

Basics

- CSS defines how elements should be rendered on screen
- Presentation of ML
- Superseded many HTML attributes that mixed presentation with content

Advantages

Separation of concern and presentation

- Speed
  - Style sheet downloaded only once
- Maintainability
  - Can be centrally maintained and easy to update
- Accessibility
  - Accessible on different devices
- Portability
  - Consistent styling across all devices
- Reduced work
  - Don't have to specify alignment for every element
- Consistency
  - Consistency in brand

Why Cascading

Going from highest priority to lower priority:

1. Inline styles
   - Applies to a single tag, defeats purpose of CSS
2. Document style sheets
   - Appear in HTML <head>, defeats purpose of separation of roles
3. External style sheets
   - Separate file and potentially on any server
   - Can be applied to any number of documents

CSS Selectors

- Determines which elements the style applies to
- Basic Selectors
  - Allow for selection based on one specific criteria
- Combinator Selectors
  - Allow one to join multiple different selector criteria

Basic selectors	Example
Universal selector	* {…}
Element selector	p {…}
Attribute selector	[secret="yes"] {…}
Class selector	.important {…}
ID selector	#1234 {…}
Pseudo-class selector	:onhover {…}
Pseudo-element selector	::first-letter {…}

Selector combinators	Example
Group selector	s1,s2,s3 {...}
Descendant selector	s1 s2 {...}
Direct descendant selector	s1 > s2 {...}
Sibling selector	s1 ~ s2 {...}
Direct sibling selector	s1 + s2 {...}

Conflict Resolution

- An element may be the subject of more than one rule because:
  1. A tag may be used twice as a selector.
  2. A tag may inherit a property *and* be used as a selector.
- This is often unavoidable as your page will invariably contain multiple style sheets with conflicting definitions:
  - Author style sheets (style sheets written and loaded by the developer)
  - User style sheets, (style sheets written by the user via the browser settings)
  - User agent style sheets (default style sheets provided by the browser)

  Precedence rules
  1. Origin and importance
  2. Judge on specificity
  3. Choose whichever selector appears last

Week 4-5: JavaScript

1. Core JS – Heart of language
2. Client-Side JS - Objects supporting browser control and user interaction
3. Server-Side JS – Objects support use in web server

Basics

- Reduce load on server by rendering client side
- User actions are events
    - Main task of JS
- Dynamic documents
- Bindings to document object model, DOM

Document Object Model

- Platform and language-nuetral interface that will allow programs and scipts to dynamically access and update the content, structure and tyle of documents
- Incorporated in present page

  DOM Tree Model
    - API describes a tree structure and reflects the document nodes in the HTML hierarchy

Browser Object Model

- DOM is a subset to BOM tree
- Includes nodes from execution environment
- Not specific to current page and no fixed standard but common set of features
    - Cookies
    - Browser
    - History
    - Screen size
    - Location

DOM in JS

Element objects addressed in several ways:

1. Type and index
    - Array like objects
    - `Document.getElementsByTagName("tag")`
    - `Document.images[0]`
2. Name
    - Attribute in HTML has `name="whatever"`
    - `Document.whatever`
    - Useful for keys that are sent as data to servers and linking with labels
3. ID
    - Unique id
    - If multiple exist with id, it creates an array
    - `Document.getElementsById("#idname")`

Persistent State in the Browser

- HTTP requests are stateless, neither server or browser maintain info after user leaves web page
    - Makes it difficult to identify users or maintain a state
- Solved by:
    - Cookies
        - Starting containing key-value pairs
        - Transmitted to server as part of HTTP
        - Small, 4KB max size
        - Access by DOM
        - Have expiry date or delete when user leaves site
    - Local Storage
        - Key-value dictionary
        - Only client side
        - Only available at the domain on which it was made
        - Larger, secure

Event Driven Programming

- Determined by sensor outputs, user actions or messages from other programs
- Fundamental to web based programming
    - Client server model
    - Stateless programming
    - Controlled via browser end
- Drives:
    - Sockets
    - AJAX
    - JS callbacks

Implementation

- Structured as program loop with:
    - Event detection
    - Event handling
- Avoid event loops as:
    - They happen through:
        1) Embedded programs -> interrupts
        2) Execution Environment implements loop -> Browser
- Event listener Registration
    1) Using Attributes
        - Simple, potentially beneficial in early development
        - Blurs lines in the separation of roles
        - HTML may load before JS
        - <button onclick=”something()”>My Button</button>
    2) Using Event Listener
        - Flexible
        - Allowing chaining of events
        - More complex to setup
        - ButtonElement = document.getElementByTagName(‘button’)
        - ButtonElement.addEventListener(‘click’, myfunction())
    3) Using Property Handler
        - Provides separation between HTML and JS
        - Can’t have multiple events for one element
        - Creation of another event on the same elements removes original event

Event Flow

- When target node is triggered for an event -> ancestors are also triggered for event
- The order in which elements receive the event is the Event Flow. Three Phases
    1) Capturing phase
        a. Rare to use
        b. Top to target node activation
    2) Target phase
        a. Target activation
    3) Bubbling phase
        a. Target to top activation
        b. Handler to ancestor not has to not be enabled

Week 6: Agile Development

Traditionally

OLD: Waterfall method:

- Project management is strictly sequenced
- One thing goes wrong -> a lot goes wrong
- Hard to determine what clients want

NEW: Agile model:

- Aims to minimise the problem when things go wrong
- Continuous improvement rather than blocking one thing off

Basics of Agile

- Breaks down large tasks into small manageable tasks call iterations
- Each iteration has a consistent time interval to produce something valuable
- Each iteration should end with client feedback

Within each iteration:

1. Sort current list of requests in terms of difficulty and priority
2. Start developing the features at top of list
3. At end of iteration, show results to end user
4. Update the list of features:
    - Removing features done
    - Add new features or reprioritise existing ones based on users feedback

How is Agile different from waterfall?

- Development is iterative
- Roles blur
- Requires change
- Cheaper overtime
- Planning is adaptive
- Working software

Approaches

Scrum, Kanban, XP, lean SD, Crystal

User stories

- Gets desired features from user stories
- Told from end user POV
- Features delivered in short units of work
- Often written ono card to facilitate communication
- Allocate story points

Time Estimation

- Hard and unpredictable

Planning

- Combine user stories and estimate time to build a feasible plan

Refactoring

- To maintain design and functionality
- Organise into manageable modules
- Refine code but preserve behaviour
- Don’t repeat yourself

Test Driven Development

- Write tests at start then write code to pass the tests
- Tests become de facto documentation

Unit Testing

- Snippets of test code, ‘codified requirements’

Continuous Integration

- Keeps code in repository, that is auto maintained and everyone works on at the same time

Summary

- Individuals and interactions over process and tools
- Working software over comprehensive documentation
- Customer collab over contract negotiation
- Responding to change over following a plan

Week 7: HTTP, AJAX

- JavaScript needs to respond to local browser events
- Need to also communicate with Server after page is loaded
    - Updating page when someone sends a message or liking a post
    - Submitting a form
- HTTP requests
- Web sockets

How is Agile different from waterfall?

- Clients send a HTTP request to server -> server receives request -> formulates and sends a response then forgets about the exchange
- HTTP protocol requests have a specific form specifying the method (GET, POST, UPDATE, DELETE) and URL, come with a header and a message body
- A response reports the status(200 - OK, 404 file not found), the header and a message body

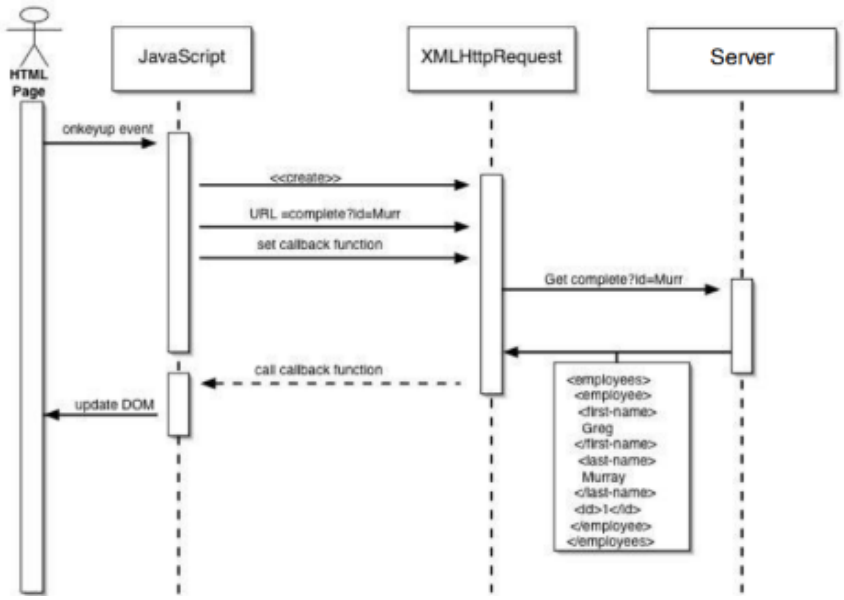
Asynchronous Communication

- When making a request, we don't know when or if the server responds
- JavaScript is single threaded so it would block other scripts from running
- The environment JavaScript runs in is not single threaded so we can:
    - Write a function which will be executed when and if the server responds

AJAX

- Update parts of a web page, without reloading the whole page.
- AJAX stands for Asynchronous JavaScript And XML
- More of an approach than technology
- Describes the information we should exchange in HTTP requests
    - Sending asynchronous http requests to a remote server and receiving structured data which could be parsed using JavaScript and dynamically update a webpage using DOM
- Server send back data in JSON or XML
    - Universal format that JavaScript can interpret
- AJAX is good as it can reduce the amount of memory and computational power the client does
    - Loading all data when the page loads VS loading only when you need it
    - Too much data to offload

Callbacks



Week 8: Client and Server Side Rendering

What is a web server

- Another computer running a program setup to respond to HTTP requests from other computers on the network
- A lot of development done through command line, since traditionally servers don't need a GUI front end

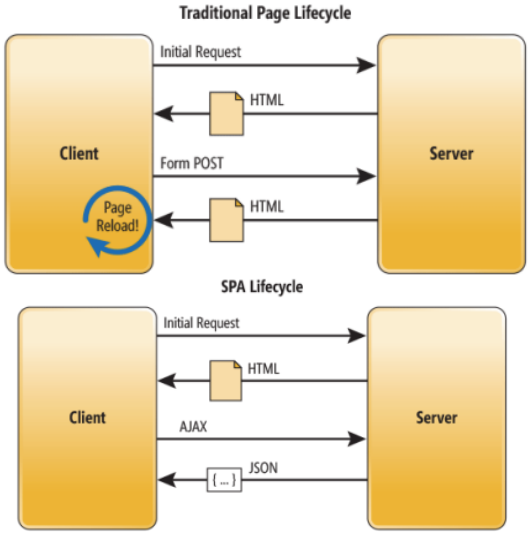
Full Stack Development

- A stack refers to a complete list of technologies required to implement both front end and a backend
- Full-stack development involves knowing a full set of technologies used for both front-end and the back-end. Most developers are specialized in one part of the stack

Server-side VS Client-side rendering

Server-side rendering is the traditional approach. However, client-side rendering is more flexible and allows greater support for non-browser devices.

1. Server-side rendering - the server builds the HTML when it receives the request and sends it to the client.
2. Client-side rendering - the server sends JavaScript and an HTML skeleton to the client, and the client can then request JSON and build the HTML using AJAX and jQuery.

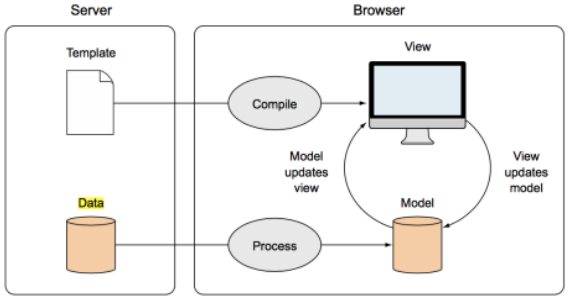


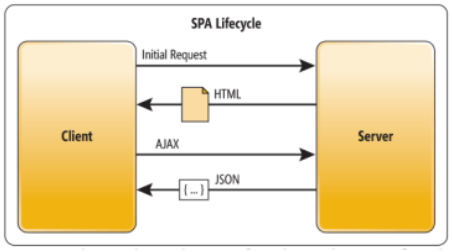
Client Side Rendering

- We often want to respond to remote events, such as someone sending you a message, liking a post etc.
- We also may want to dynamically respond to a local event using information on the server
- We could send the date to the server, have the server rebuild the page and send the entire page back, but we only require a few bytes of data

Single page applications (SPA)

- Single Page Applications are services where the entire website is provided via client-side rendering:
    - The browser/client do the heavy lifting i.e. logic and rendering.
    - The server just provides the data.
    - The user never navigates to a new URL, even when they move to what looks like a new page.





Advantages and Disadvantages

Pros of SPA

- Less load on the server, able to respond to more clients.
- A more responsive client. No need to wait for server responses.
- Genuine separation between content and presentation.

Cons of SPA

- Longer load time. A lot of JS must be transferred.
- Search engine optimisation (SEO) can be a problem. Robots won't crawl JavaScript.
- Navigation (e.g. forward and back buttons) can be an issue.

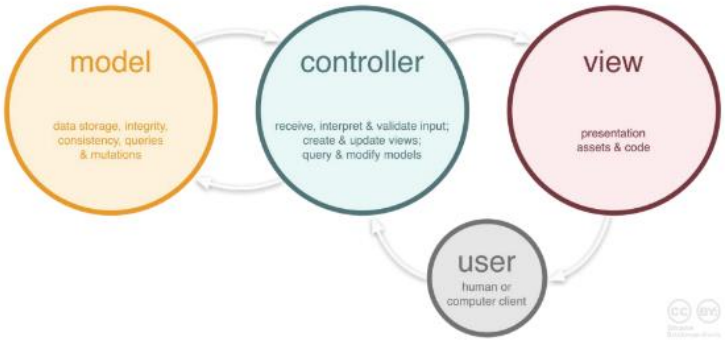
Week 9: Databases

Architecture

- Design patterns describe re-useable design concepts, particularly in software. They describe how concepts are organized.

Model View Controller Pattern

The model-view-controller(MVC) pattern is one of the most popular architectures for server-side web applications.



Advantages of MVC

- The division of the web-application in this manner has several advantages:
  - You can alter the view without altering the model.
  - You can swap the database out without altering the view.
  - Specialist developers can work on each bit.
  - Can support multiple views (e.g. web page + mobile app).
  - Easier to test parts in isolation.

MVC in Agile Web Dev

- The model refers to the Python objects in the application backed by the database (SQLite).
- The view are the HTML pages created from the Jinja templates.
- The controller is the code in routes.py that
  i. links the model to the database
  ii. prepares the view based on the model
  iii. the updates and saves the models back to the database.

Designing an MVC architecture

- To design an MVC solution architecture, you need to identify what models, views and controllers you require.
- Recall user stories are simple representations of software requirements.
- In every user story, we can identify nouns which could be models, verbs which could be routes, and associate a view for the specified user.
- We can then mock-up wireframe sketches of view and mock HTTP requests and responses.
  - Mock up views are sketches of the basic layout and functionality of a UI
  - Show the sequence of interfaces used in an application
- Mock up APIs:
  - You can also mock the typical HTTP requests and responses your app will serve
  - These can be hard coded using tools like Apiary and Mocky (more on this later)

Implementing models

- A model is an object that is paired with an entity in a database.
- There is an Object Relational Mapping (ORM) linking the data in the database to the models in the application.
- The models are only built as needed and update the database as required. Most frameworks include ORM support.
- To build the models, we first need to set up the database.

Databases

- ORM is the description of the data
- CRUD system

We will use the relational database SQLite as our DBMS because:

- Simple to setup
- Small package size
- Stores the database as a single file

Disadvantages compared to MySQL:

- Doesn't scale well
- Doesn't have inbuilt authentication methods.

Real life web servers:

When your app reaches a certain size, your database must support many more things. This includes:

- Adding indexes to allow more performant queries on frequently used data.
- Optimising data layout depending on whether it is most often read from or written to.
- Caching to avoid performing the same queries repeatedly.
- Security to stop malicious users or applications accessing your database even if they compromise the server.
- Backups to avoid losing your database if your server fails.
- Auditing to keep track of when a database was altered and by whom.

Week 10: Security

Security on the Web

- Primary concern for web applications
- Work under the assumption that the network is compromised
- Data access must be controlled, passwords must be validated securely and users just be able to trust the information presented to them
- Complete security is hard to achieve

Breakdown

Security on the web depends on trust. There are several elements to this:

1. Web server need to be confident that someone accessing data is authorised
    - Achieved via session/token-based authentication
2. User need to know that the site they are visiting is the one they intend to go to
    - Achieve via SSL certificate
3. Both Server and client need to be confident that there is no middle man accessing the data
    - Achieved via HTTPS (HTTP Secure)

These are all achieved by public-key encryption and cryptographic hashing

Public key encryption

Secure communication is based on Public Key Encryption (e.g. RSA). There are two functions, Ek (•) - encrypts with key k, and Dk (•) - decrypts with key k.

- A public private key is a pair of keys pub and priv such that
    - x = $D_{priv}(E_{pub}(x))$
    - x = $D_{pub}(E_{priv}(x))$
- So you can't work out X even if you know pub

Various uses of Public Key encryption:

- Authentication
    - The value of $E_{pub}(x)$ can be published and only someone who knows priv can work out what x is by computing $D_{priv}(E_{pub}(x))$
    - Encryption is public but decryption is private
- Digital Signatures
    - The pair (x, $E_{priv}(x)$) can be verified by anyone by computing $D_{pub}(E_{priv}(x))$ and comparing it to x, but only created by someone who knows priv.
    - Person privately creates x but public can't decrypt
- Key distribution
    - A new random key x can be generated, and $E_{pub}(x)$ can be sent to someone who knows priv. The pair then knows x, but nobody else does
    - Person publicly creates x and then only the server with priv can decrypt

Crypto graphic hashing (MD5)

- Good at Secure data storage
- A cryptographic hash function is a function hash that takes a string or arbitrary length and returns a fixed length integer such that:
    - Given a value h it is difficult to find an x such that hash(x) = h
    - Given a value x, it is difficult to find a y such that hash(x) = hash(y)
- computes a number from a stream of data, in such a way that it's very difficult to fabricate data with a certain hash
- Useful for Storing passwords securely
    - given password p, you can store h = hash(p) in your database instead of p and when the user tries to login with password q, you can test whether hash(q) = h

Session-based user authentication

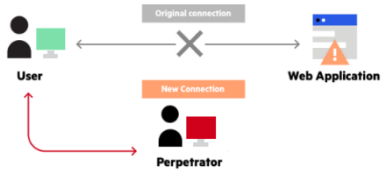
- Although HTTP is stateless, the application is not
- To track a users session, we store their username and password. Password is salted and hashed
- When logging in again, they provide the same password

    - This is again salted and hashed and compared to the hash in the data base
- If successful, the server's response contains a cookie with the session ID and hash(ID + secret)
- Whenever a future HTTP request is made, the server checks the session ID in the cookie and recomputes hash(ID + secret) to check the ID hasn't been altered

Attacks on Session-Based Protocols

Impersonation attacks

Sessions allow the server to authenticate the user. However, remember the network is still fundamentally insecure

- Without extra security, the easiest attacks is an impersonation attack
    - The network redirects the user's request to a fake server
    - The fake server acts just like the real server
    - User submits, passwords, private data, bank details ect. To the fake server
    - 

Prevent with SSL Secure Socket Layer

An impersonation attack can be prevented by the server a certificate that it's real via the Secure Socket Layer (SSL) protocol. Certificates are provided by a trusted third party known as a Certificate Authority (CA).

- When a server requests a certificate it sends their public key s_pub to the CA, who, after various checks, returns (s_pub, Ec_priv(s_pub)) where c_priv is the private key of the CA.
- When a user contacts the server, the server sends them (s_pub, Ec_priv(s_pub)), and the user can then use c_pub, the CA's public key, to compare s_pub to Dc_pub(Ec_priv(s_pub)).
- If it matches, then the user knows the CA has vouched that the server has the public key s_pub, and only the real server knows the private key s_priv and therefore can respond to messages encrypted with s_pub.
- What's not solved: how do you trust the CA is real? Answer: root certificates
    - Trust in your browser

Man in the middle attacks

Even with using certificates, network can still perform man-in-the-middle attack

- All traffic is redirected through attacker and forwarded to the real server
- Because the user is interacting with the real serve, certificates pass the checks
- However, the attacker still gets access to all the information sent back and forth. Including signed cookies

Prevent with Encrypted Sessions with SSL

Can't prevent the network from redirecting the messages -> must encrypt messages

- SSL also includes a public key encryption process to enable secure HTTP requests
- After initial certificate handshake establishing that the server has the public key
- Both parties use a key distribution protocol to generate a shared set of private session keys
- They then use these keys to encrypt all future traffic between them during that session

HTTPS

- SSL, in theory, can be used to encrypt any data between two end points
- Today most request are still HTTP
- Modern browsers highlight if the website is HTTPS to motivate adoption

Cross-site request forgery (CRSF) attack

- Even using certificates and HTTPS doesn't make your application safe
- CRSF exploits the fact that users may already be authenticated to a server
- Suppose you've recently logged in to your bank at www.mybank.com and therefore have a signed cookie from their server sitting in your browser
    - Then you receive an email and it has a link and you click on it
    - The actual link is something like this:

      www.mybank.com/transfer?from=savings&destBSB=112043&destNo=1230111&amount=500.00

- Remember that cookies are transferred automatically when making the request to the domain they are associated with, and therefore the signed cookie is sent to the server
- The server checks the cookie, which passes as it is real, and therefore thinks the request is from the user, and the transfer goes through!

Defending against CRSF

- Server has a secret key which it renders on the form
- When the form is submitted, the secret key is checked during validate_on_submit
- Server will only respond to requests generated via an official form
- Safety critical apps will often time-out signed cookies after ~5 minutes

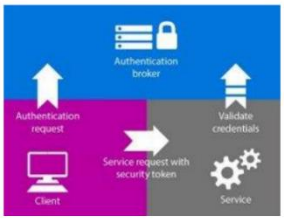
Token-based user authentication

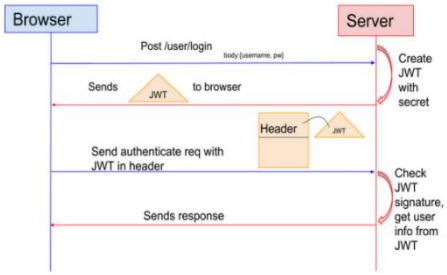
Downsides of session-based authentication

- Works but has drawbacks
  1. Requires the application to track all user session -> may not scale well
  2. HTTP requests are sent in plain text -> passwords should never be sent in plain text
  3. User may be on a mobile where there is no such thing as a cookie
  4. Inherently stateful -> may not be necessary for API access
  5. Requires users to register their info with the server
- What if we only want to check authorisation rather than authentication???

Tokens and JWT

- Same as cookies but not used to authenticate a user and retrieve the state on the server
- Instead they Authorise the holder to perform a particular action
- Not necessary for server to know who requester is
-





Advantages of tokens

- Sacrificing the ability to store state, tokens have many adv over sessions
  1. Validating a token doesn't require querying the database.
  2. Tokens aren't vulnerable to CRSF attacks (as long as they are not stored in cookies!).
  3. They don't require cookies and therefore work on mobile apps
  4. Tokens can store a set of privileges, rather than just one
- Tokens are also often issued by trusted third parties instead of the server itself
  - the server doesn't have to store sensitive user information
- obtaining the token still usually requires authentication via a password protected user account! Therefore, state is still required, but only at the beginning.

OAuth

OAuth was developed by Twitter to allow applications to authenticate and interact with Twitter, without requiring repeated logins.

- OAuth verifies a user's identity and provides a JSON Web Token (JWT). This contains a header, a payload and a signature in compressed JSON.
- This header describes the encryption type, the payload typically provides some user identifier, an expiry and issuer information, and the signature is a secure hash, proving the token wasn't tampered with

Cross-site Scripting

Unexpected inputs to escape the current statement and begin executing arbitrary code

- SQL injection

Defending with Sanitising User Inputs

- Automatically escape any control characters present in the string
  - Using / like /n
  - In SQL this involves inserting a backslash before a character.
- SQLAlchemy
- Jinja automatically escapes any strings substituted into HTML templates
- Other approaches include input validation, prepared queries

Week 11: Testing

Making application bug free

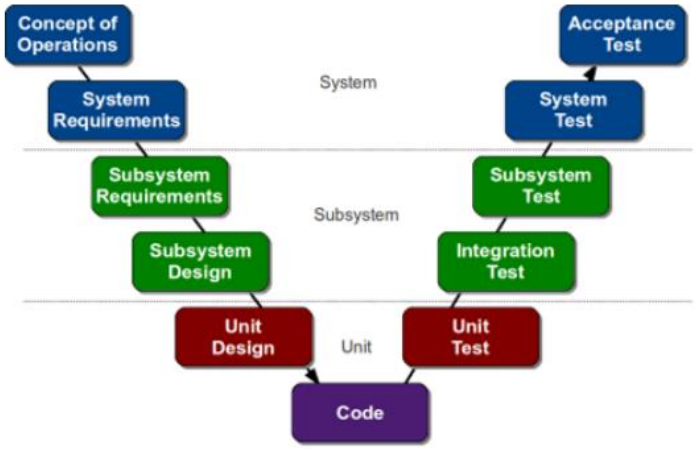
- Bug-free is critical to success
- Various methods include:
    1. Code reviews : Having peers critically examine your code and make suggestions
    2. Testing: Providing test cases of inputs and actions and expected behaviours
    3. Formal verification: building precise specifications of correctness, and proving the code meets these specs

Types of tests

- Testing is a key activity in any software development, but particularly in Agile development
- Test suites are a proxy for requirements documentation
- Agile relies heavily on test automation, so that every sprint or iteration can be checked against the existing test suite
- There are many different categories. One way is as follows:

1. Unit tests: Test an individual function to ensure it behaves correctly.
2. Integration test: Execute each scenario to make sure modules integrate correctly.
3. System test: Integrate real hardware platforms and test their behaviour.
4. Acceptance test: Run through complete user scenarios via the user interface.

V-model of Testing

- The V model links types of test to stages in development process
    - We only focus on unit and system tests
    - 

Unit Tests

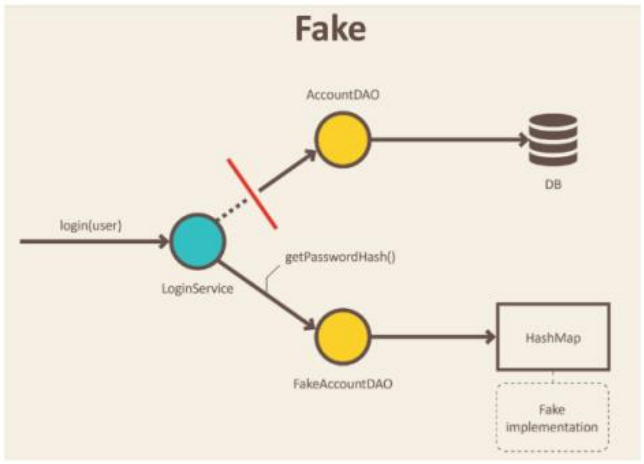
Purpose is to test an individual function to ensure it behaves correctly

- Usually 2-5 unit test are required per function to cover all edge cases
- Properties include
    1. It should be automated – running it should not require a human in the loop.
    2. It should be repeatable – it should not update persistent state, e.g. external database.
    3. It should run quickly – there will be many unit tests, so each one should be fast.
    4. It should pinpoint the failure – if it fails, you should be able to find the problem.
    5. It should be limited in scope - any changes to anything outside the function should not affect whether the test for that function passes.
        i. To address the last point, in order to isolate the system under test (SUT) from external systems, we use test doubles: fakes, stubs and mocks.

Test Doubles

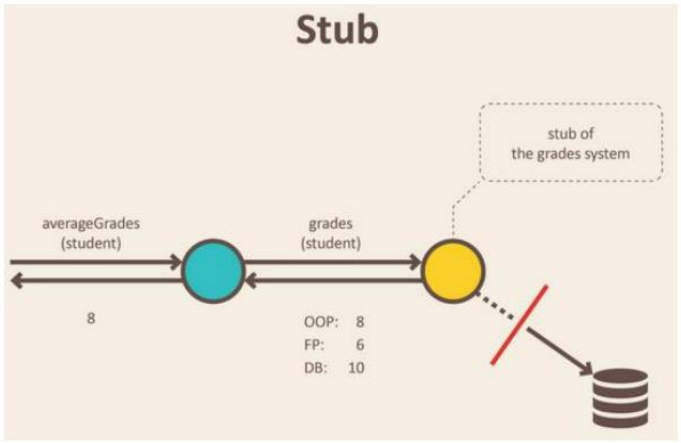
Fakes

- Object with working implementations, but not the same as the production environment

- 

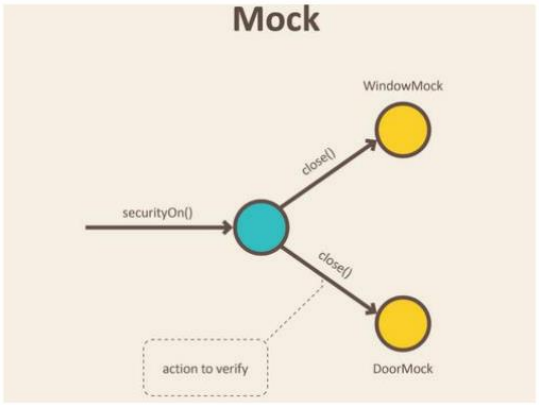
  - In the diagram, the login method is being tested, but the full database has been replaced by a fake - an object wrapping a HashMap.

Stubs

- Stubs are objects that holds predefined data to respond to specific requests, but do not provide the full functionality
  - Requests that return errors or a set variable that would have been allocated by users
- 

  - In the diagram, a stub database system only provides a fixed pre-determined set of grades for each student, that are then used to test the averageGrades function.
  - As another example, to test the login GUI, we could provide a stub that accepts only the password ‘pw’ regardless of the user.

Mocks

- Mocks work like stubs, but they remember the calls they receive, so we can assert that the correct action was performed, or the correct message was sent
- 

  - In the diagram, door and window mocks are used to verify that the close() method was called, without interacting with hardware

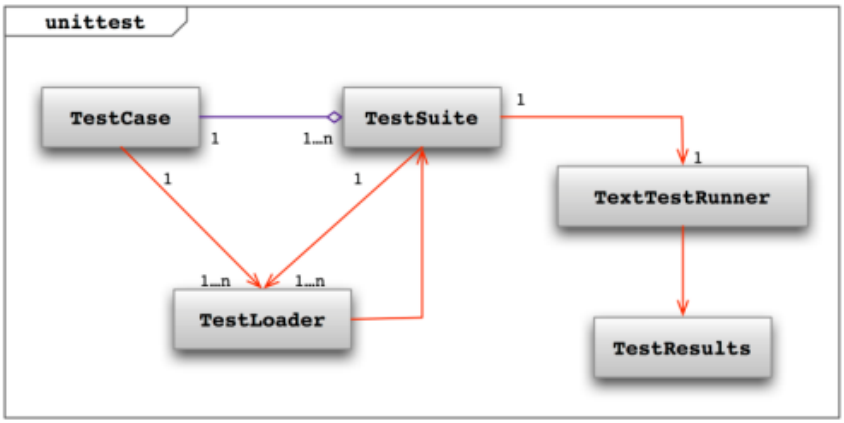
Unittest Module

In Python, unit testing is commonly done with the module unittest, which provides several classes and functions;

- Test fixtures: These are the methods to prepare for a test case, called setUp and tearDown which run before and after each test.

- Test Case: This is the standard class for running a test. It specifies the test fixtures, and the functions to execute.
- Test Suite: Running comprehensive tests is expensive, so often you don't want to run every test. Test suites allow test cases to be grouped together to be run at once.
- Test Runner: The class responsible for running the tests and report the results.

Typically, you only need to write the test cases, and the rest is automatic.



System Tests

System tests integrate real hardware platforms and test their behaviour. They are more challenging to write than unit tests since they depend on the end user environment

- In the web, this means testing the behaviour of how the application works in a browser

Selenium is one possible program that can be used to automate browsers to run test cases. It has two variations:

- Selenium IDE is a browser plugin that can record interactions with a web-site and run them back to confirm the outcome remains the same.
- Selenium WebDriver provides a set of tools for scripting system tests
- Selenium IDE saves all information in a table form.
    - Each record consists of:
        - Command – tells Selenium what to do (e.g. "open", "type", "click", "verifyText")
        - Target – tells Selenium which HTML element a command refers to (e.g. textbox, header, table
        - Value – used for any command that might need a value of some kind (e.g. type something into a textbox)

The advantages of Selenium IDE:

- Useful for quickly prototyping tests.
- Useful for creating a bug replication report that others can use.
- Can visualise the test.

The disadvantages of Selenium IDE:

- Difficult to maintain tests, e.g. if the page changes, you must re-record the whole test.
- You can't apply test fixtures (i.e. setUp and tearDown) easily
- You need a running instance of the browser

To fix this use the **Selenium WebDriver**

Tests as specifications

Testing is essential for reliable software:

- We would ideally like to have a "complete" set of test cases, such that any code that passes the test "works".
- Logically, this means that any line of code that does not get executed in at least one test case is redundant to your notion of "works".
- The proportion of the lines of code that are executed in your test suite is known as the code coverage.
- This informs the philosophy of Test-Driven Design (TDD) where tests are written first, and the code designed specifically to pass those tests.

- There are different ways of measuring coverage: statement coverage, branch coverage, logic coverage, path coverage. Statement coverage is sufficient for our purposes, but you should always consider the ways your tests may be deficient.
- Coverage can be automatically measured by such tools as the coverage Python package, and HtmlTestRunner can be used to give visual feedback on a test run.

Week 12: RESTful APIs

Web APIs use HTTP requests, in standardized formats with documented response types.

- The most common API design is that based on REST APIs.
- Provide the logic and data structures of your app as a service to other developers so they can embed the functionality into different applications and customise the user interface
-

Representational State Transfer

HTTP is stateless, so there is no memory between transactions. REST uses the current page as a proxy for state, and operations to move from one to the other.

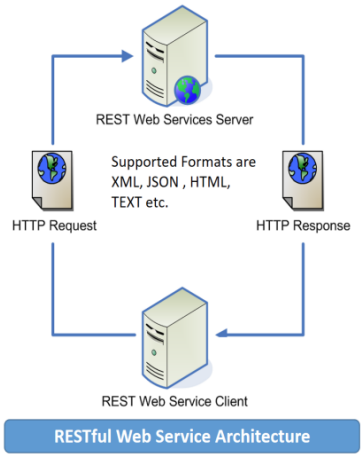
The six Characteristics of REST

1. Client Server model

The client-server model sets out the different roles of the client and the server in the system.

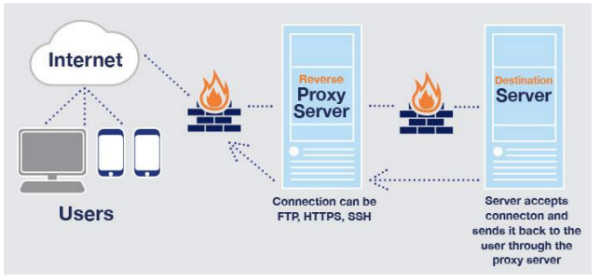
They should be clearly differentiated and running as separate processes and communicate over a transport layer.

In practice the interface between the client and the server is through HTTP, and the transport layer is TCP/IP.



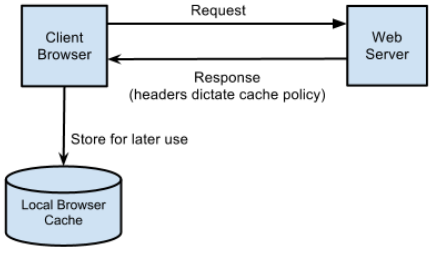
2. Layered System

- The layered system characteristic states that there does not need to be a direct link between the client and the server, and that they can communicate through intermediate nodes.
- The client does not need to distinguish between the actual server and an intermediary, and the server doesn't need to know whether it is communicating directly with the client.
- This encapsulates the abstract nature of the interface, and allows web services to scale, through proxy servers and load balancers.

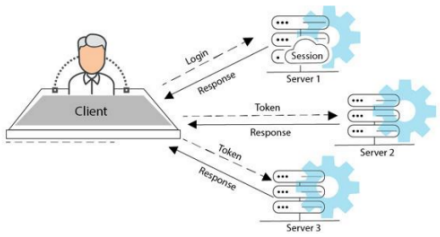


3. Cache

- The cache principle states that it is acceptable for the client or intermediaries to cache responses to requests and serve these without going back to the server every time.
- This allows for efficient operation of the web.
- The server needs to specify what can/can't be cached, (i.e. what is static and dynamic data).
- Also, anything encrypted cannot be cached by an intermediary.
- All web browsers implement a cache to save reloading the same static files.



4. Code on Demand
    - The code on demand principle states that the server can provide executable code in responses to a client.
    - This is common practice with web browsers, where JavaScript is provided to be run by the client.
    - However, this isn't commonly included in REST APIs since there is no standard for executable code, so for example, iOS won't execute JavaScript.
5. Stateless
    - Statelessness is a key property of the HTTP protocol, and most associated with REST APIs.
    - Servers should not maintain any memory of prior transactions, and every request from clients should include sufficient context for the server to satisfy the request.



    - The *representative state* is in the URL or route that is requested by the client and is sent through with each request.
    - This makes the service easy to scale, as a load balancer can deploy two servers to satisfy arbitrary requests, and they do not need to communicate.
    - Pragmatically, many REST APIs do record state for session management.
6. Uniform interface
    - The most important, and most vague, requirement of REST is that there be a uniform interface, so clients in principle do not need to be specifically designed to consume a server.
    - The four aspects of the uniform interface are:
        - Unique resource identifiers. This is the URL, and typically is of the form

          `api/users/<id>`

        - Resource representations. The data exchange between client and server should be through an agreed format, typically JSON, but possibly others. HTTP can do content negotiation.
        - Self-descriptive messages. The communication between client and server should make the intended action clear.
        - Hypermedia links. A client should be able to discover new resources by following provided hyperlinks.

REST API CRUD operations

HTTP METHOD	CRUD	ENTIRE COLLECTION (E.G. /USERS)	SPECIFIC ITEM (E.G. /USERS/123)
POST	Create	201 (Created), 'Location' header with link to /users/{id} containing new ID.	Avoid using POST on single resource
GET	Read	200 (OK), list of users. Use pagination, sorting and filtering to navigate big lists.	200 (OK), single user. 404 (Not Found), if ID not found or invalid.
PUT	Update/Replace	404 (Not Found), unless you want to update every resource in the entire collection of resource.	200 (OK) or 204 (No Content). Use 404 (Not Found), if ID not found or invalid.
PATCH	Partial Update/Modify	404 (Not Found), unless you want to modify the collection itself.	200 (OK) or 204 (No Content). Use 404 (Not Found), if ID not found or invalid.
DELETE	Delete	404 (Not Found), unless you want to delete the whole collection – use with caution.	200 (OK). 404 (Not Found), if ID not found or invalid.

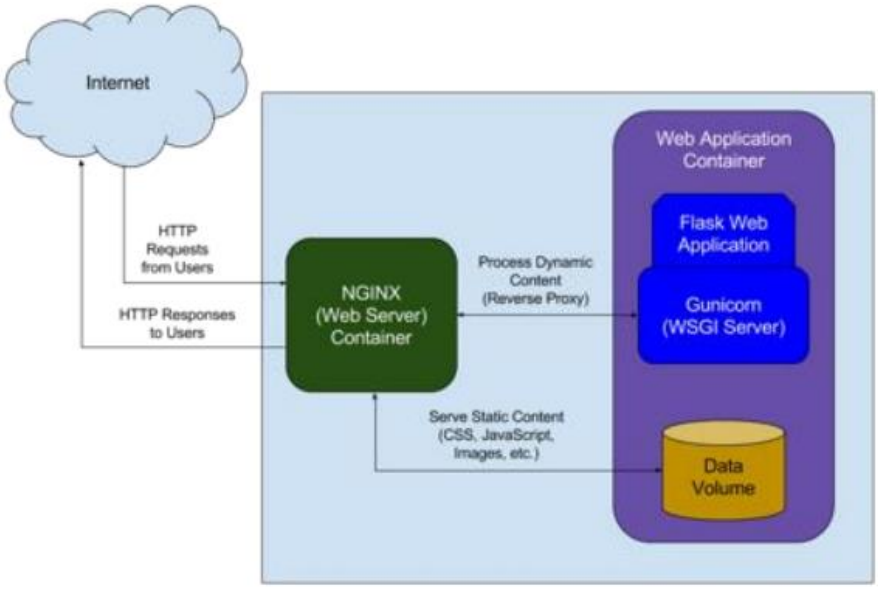
Week 12: Deployment

Industrial strength servers, and consider deploying via: 1. A Linux virtual machine 2. Your own server 3. Heroku.

Replace SQLite with MySQL as MySQL is a server database rather than an embedded database

Replace Flask with gunicorn

Make reverse-proxy server with NGINX



Future of Web

- Cloud computing
- Internet of Things
- The semantic Web
- Cyber Security